netcheck

Release 1.10.0

Gene C

Jan 04, 2026

CONTENTS

CHAPTER

ONE

NETCHECK

1.1 Overview

netcheck : Check network connectivity

1.2 Key features

- netcheck uses ping to check network status of multiple hosts/ips
- Checking a single remote site is insufficient to distinguish whether the path to that remote is the only route unavailable, or if all routes are down. We therefore recommend checking 2 hosts simultaneously. While more may be better, we have found using 2 reliable remote hosts eliminates all false negatives.
- Data is saved to file(s)
- netcheck-plot can be used to plot the saved data.

1.3 New / Interesting

1.10.0

- Switch packaging from hatch to uv
- Testing to confirm all working on python 3.14.2
- License GPL-2.0-or-later

1.4 Getting Started

netcheck is designed to be run periodically, saving results to files from which netcheck-plot can be used to create plots showing packet loss over time.

By default netcheck is silent unless there is packet loss. This is convenient when run from cron.

Using the *-v* option allows it to show output even when no packet loss is seen.

For example you may want to run this command from cron, perhaps every 15 minutes:

```
netcheck -d /opt/netcheck 1.1.1.1 8.8.8.8
```

it will report nothing unless both remote hosts have some packet loss. Since occasional packet drops are not uncommon, its good to have 2 or more hosts for the check. You can use *–any* option to have it report if any host has some packet loss.

If no hosts are provided, then default hosts are *1.1.1.1* and *8.8.8.8*.

Using the *-d <dir>* option has packet loss data saved into files under the specified directory. With the above command observed packet losses will be saved to 4 files:

```
ncheck-1.1.1.1
ncheck-8.8.8.8
ncheck-merged.any
ncheck-merged.all
```

One file for each host and the *ncheck-merged.any* records packet seen by any host and *ncheck-merged.all* has the data when all hosts had packet loss seen at the same time.

The *-I, –Interface* option can be used to specify which network interface the check is to be run from. Not to be confused with the *-i, –interval* which specifies the interval used between each ping packet.

To generate a plot for any of the files simply run:

```
netcheck-plot -f /opt/netcheck/ncheck-merged.all
```

which by default will create the pdf file

```
./pdf/ncheck-merged.all.pdf
```

For all available options please see *–help* for either application.

1.4.1 How It Works

Netcheck runs ping in parallel on each host listed using asyncio. By default 100 packets are sent to each host, though this can be changed using the *–num* option.

1.5 Options

Available options from *–help*:

netcheck:

```
positional arguments:
  hosts                 host(s) comma separated if more than one (None)

options:
  -h, --help            show this help message and exit
  -a, --any             Report error if any host has loss instead of all hosts (False)
  -b, --base BASE       base name for output (ncheck)
  -d, --dir DIR         directory to save output ()
  -i, --interval INTERVAL
                        ping interval (0.25)
  -I, --Interface INTERFACE
                        ping interface or IP address to use as source ()
  -4, --ip4-only        Use IPv4 only (False})
  -6, --ip6-only        Use IPv6 only (False)
  -n, --num NUM         num pings (100)
  -np, --no_parallel    No parallel (False)
  -t, --test            test mode - generate fake lost packets(False)
  -v, --verb            verbose mode (False)
```

netcheck-plot:

```
options:
  -h, --help          show this help message and exit
  -d, --dir DIR       Directory to write plot pdf file (./pdf)
  -e, --end END       End time : absolute or relative days (all)
  -f, --file FILE     Input csv file (data)
  -o, --out OUT       Output PDF : if not specified will be based on input filename
  -s, --start START   Start time : absolute or relative days (all)
  -sh, --show         Show plot in terminal (False)
```

CHAPTER

TWO

APPENDIX

2.1 Installation

Available on * Github * Archlinux AUR

On Arch you can build using the provided PKGBUILD in the packaging directory or from the AUR. All git tags are signed with arch@sapience.com key which is available via WKD or download from https://www.sapience.com/tech. Add the key to your package builder gpg keyring. The key is included in the Arch package and the source= line with *?signed* at the end can be used to verify the git tag. You can also manually verify the signature

```
git tag -v <tag-name>
```

To build manually, clone the repo and :

```
rm -f dist/*
/usr/bin/python -m build --wheel --no-isolation
root_dest="/"
./scripts/do-install $root_dest
```

When running as non-root then set root_dest a user writable directory

2.2 Dependencies

Run Time :

- python (3.13 or later)
- python-pandas
- python-matplotlib
- pyconcurrent

Building Package :

- git
- hatch (aka python-hatch)
- wheel (aka python-wheel)
- build (aka python-build)
- installer (aka python-installer)
- rsync

Optional for building docs :

- sphinx
- texlive-latexextra (archlinux packaguing of texlive tools)

2.3 Philosophy

We follow the *live at head commit* philosophy as recommended by Google's Abseil team[1]. This means we recommend using the latest commit on git master branch.

2.4 License

Created by Gene C. and licensed under the terms of the GPL-2.0-or-later license.

- SPDX-License-Identifier: GPL-2.0-or-later
- SPDX-FileCopyrightText: © 2025-present Gene C <arch@sapience.com>

[1] https://abseil.io/about/philosophy#upgrade-support

CHAPTER

THREE

LICENSE

Netcheck software license.